Yujia Zhang
May 7, 2026

Exercise Solution

Summary: *Predicting exact home and away corners is hard, so the model should not stop at point forecasts. The useful step is to combine home and away means, variances, and market-aware calibration into a betting distribution.*

Once that distribution is good enough, Kelly can compound capital even under realistic fees and stake caps, but full Kelly is too aggressive in practice. Fractional Kelly is more sensible because it gives a better risk-adjusted profile, which matters more here than maximising terminal cash. Just as importantly, Kelly is highly sensitive to probability error, so stress testing is not optional.

Questions 2 and 3 are standard at the formula level: EV, distributions, and Kelly are well known. The real issue is whether the model probabilities behind those formulas are believable. That is why the assumptions have to be checked, especially with Monte Carlo. If the EV is not credible, Kelly is not credible either.

Question 1. Build a predictive model for the number of home and away corners. Explain the process - including data exploration, features and model selection/comparison, model validation etc?
Answer. The Q1 path is:

`data_cleaning.ipynb` → `q1_pipeline.py`

Inside `q1_pipeline.py`, Stage 1a calls `team_strength.py` and Stage 3 calls `team_variance.py`. In practice the order is:

1. `data_cleaning.ipynb` does the data preprocessing.
2. `q1_pipeline.py` loads those parquet files and builds the full Q1 dataset.
3. Stage 1a in `q1_pipeline.py` calls `team_strength.py` to build leak-free pre-match team strength features.
4. Stage 1b builds rolling recent-form features from earlier matches only.
5. Stage 2 fits the mean models for home corners and away corners.
6. Stage 3 calls `team_variance.py` to estimate conditional variance, then applies the late-window correction, market-teacher adjustment (no odds are used in this stage), and residual quantile calibration.

So `q1_pipeline.py` is the orchestrator, while `team_strength.py` and `team_variance.py` are helper modules that produce two of the most important feature blocks.

Data exploration:
See data_cleaning.ipynb. It reads the raw CSV files, removes outliers (999 in corners data csv) and conflicting records, only keeps the first occurrence of the duplicate records. It also removes < 1 odds for oh and oa, and negative od odds in "OU" and "1X2". After done these steps, 7 team's corner records are missing from corner data. These are filled by matching the records from the price tables (market odds are not involved in this stage). The clean tables are written to `train.parquet`, `betting.parquet`, and `all_matches.parquet`.

What does Q1 actually predict for one match, and where do mean and variance enter?
Q1 makes two different kinds of prediction for each match.

1. A **point forecast**: how many corners the home team is expected to win, and how many corners the away team is expected to win.

2. An **uncertainty forecast**: how much those two point forecasts are expected to vary around their centres.

The point forecast is simply the pair

$$\hat{\mu}_{home} = \texttt{pred\_home\_corners}, \qquad \hat{\mu}_{away} = \texttt{pred\_away\_corners}.$$

If the model says 5.8 and 4.3, then Q1's direct answer for that match is "home about 5.8 corners, away about 4.3 corners." The variance does not create a second point forecast and it does not shift the mean. It only says how uncertain the model is around those centres.

After the two means are predicted, I estimate one variance for the home prediction and one variance for the away prediction (later used in Q2). So the final Q1 output is:

$$\hat{\mu}_{home},\ \hat{\mu}_{away},\ \hat{\sigma}^2_{home},\ \hat{\sigma}^2_{away},$$

plus residual quantile anchors $q10, q50, q90$ that will be also used by Q2.

To predict the number of corners, I introduced a team-strength model as a sub-model of mean prediction, since team strength is a key determinant of corner output.
The team-strength block turns raw match history into a running estimate of each team's latent corner profile. It is best described as an **Elo-style online updater**, but it is not the classical Elo system used for win/loss results.

In classical Elo, each team has one rating and the update is driven by whether the team won or lost. Here, each team carries two hidden states:

- an attack rating: how much the team tends to create corners;
- a defence leakiness rating: how many corners the team tends to allow.

The model walks through matches in chronological order. Before updating anything, it reads the current ratings and builds the features for that match. Only after the match is observed does it update the two teams' ratings. That detail matters, because it means every row only uses information that was available *before kickoff*. There is no look-ahead leakage.

The core abstraction is:

$$\log(\lambda_{home}) = \text{league home baseline} + \text{home attack} + \text{away defensive leakiness},$$

$$\log(\lambda_{away}) = \text{league away baseline} + \text{away attack} + \text{home defensive leakiness}.$$

Here λ_{home} and λ_{away} are the expected home and away corner rates before the match starts. After the match ends, the model compares the realised corners with these expected rates on the log scale and nudges the two teams' latent ratings up or down. Teams with little history move faster. Teams with long history move more slowly. That is what the prior-strength and learning-rate terms are doing.

So the Elo idea appears in the mechanics:

- keep a running latent rating per team;
- compare expectation with what really happened;
- update the ratings after each match;
- make updates smaller once the rating is supported by more history.

What changes is the target being updated. It is not the final match result. It is the team's corner-creation and corner-concession behaviour.

This block exports the following pre-match features:

- overall attack rating and overall defensive leakiness for both teams;
- home-only and away-only venue splits;

- implied home and away log corner rates;
- overall strength difference;
- venue-adjusted strength difference.

The block answers: how strong is each team at winning corners, how weak is it at preventing them, and how much of that signal changes between home and away. These Elo-style outputs are not used as stand-alone predictions. They are fed into the mean model as part of the feature vector. In other words, the LightGBM model looks at variables such as `home_attack_rating`, `away_defence_leakiness`, `log_lambda_home`, and `strength_diff`, then learns how those pre-match strength snapshots map into actual home and away corner counts.

What features go into the corner prediction model? How are they constructed?
The final production version uses 28 mean features. They fall into four groups.

Group 1: match context.

- `season_id`
- `gameweek`

These are cheap but useful controls. Season catches slow structural shifts in the data. Gameweek gives the model a simple place to learn season-phase effects.

Group 2: team-strength features from Stage 1a.

- `home_attack_rating`
- `home_defence_leakiness`
- `away_attack_rating`
- `away_defence_leakiness`
- `home_attack_at_home`
- `home_defence_at_home`
- `away_attack_at_away`
- `away_defence_at_away`
- `log_lambda_home`
- `log_lambda_away`
- `strength_diff`
- `venue_strength_diff`
- `league_log_baseline_away`

These features summarise the current pre-match corner environment.

Group 3: recent corner form.

- exponentially weighted averages of corners won and corners conceded;
- rolling-window standard deviations of corners won and conceded.

The idea is simple: recent corner flow tells us whether a team is currently producing pressure or absorbing pressure, and recent standard deviation tells us whether that pattern is stable or noisy.

Group 4: recent goal state.

- rolling-window goals scored;

- rolling-window goals conceded;
- rolling-window goal difference.

These do not predict corners directly, but they help because style and game state matter. Teams playing on the front foot tend to create different corner profiles from teams sitting deep or chasing games.

All rolling features are built from a per-team match table. One original match becomes two rows:

- one row from the home team's point of view;
- one row from the away team's point of view.

For each team row, I record:

- `corners_for`: corners won by that team in that match;
- `corners_against`: corners conceded by that team;
- `goals_for`: goals scored by that team;
- `goals_against`: goals conceded by that team;
- `goal_difference`: goals scored minus goals conceded.

Then I compute rolling and exponentially weighted summaries using only earlier rows from the same team. That is what keeps this block leak-free.

The 28-feature version keeps Q1 at least as good as the larger model, and it is much better when passed into Q2. That is why I kept 28 and did not drop further.

Top signals are:

- `season_id`
- `away_corners_against_ewm_half_life_5`
- `home_goal_difference_rolling_window_20`
- `home_corners_for_ewm_half_life_5`
- `away_goal_difference_rolling_window_20`

Why use LightGBM for the mean model instead of Ridge, Random Forest, or XGBoost?
I did not want to claim that by intuition alone, so I ran the comparison in code. On the Q1 validation split:

Model	Home MAE	Away MAE	Diff MAE
LightGBM	2.2777	1.9866	3.3489
RandomForest	2.2866	1.9932	3.3570
Ridge	2.2876	1.9633	3.3346
XGBoost	2.3194	2.0051	3.4007
Mean baseline	2.3906	2.0909	3.5652

The baseline is the mean of the training set predictions. It gives:

- home MAE = 2.3906;
- away MAE = 2.0909;
- corner-difference MAE = 3.5652.

Here MAE is

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i| .$$

So home MAE is the average absolute error in predicted home corners, and away MAE is the same quantity for away corners. The best possible value is 0. There is no universal finite upper bound in theory, because the target is a nonnegative count and a model can be arbitrarily wrong. In practice the scale is anchored by the observed corner range in this dataset, so a move of about 0.1 in MAE is meaningful.

The current 28-feature LightGBM Q1 model gives:

- home MAE = 2.2777;
- away MAE = 1.9866;
- corner-difference MAE = 3.3489.

So the gain over baseline is:

- home MAE improves by 0.1129;
- away MAE improves by 0.1043;
- corner-difference MAE improves by 0.2163.

That is not a huge jump, but it is real, stable, and achieved without leaking holdout information into the mean head. The bigger practical win is that Q1 is not only a slightly better point predictor than the constant mean baseline. It also exports a much richer distributional object that Q2 can actually bet on.

The procedure of prediction:
The final prediction is assembled in layers.

1. Fit one mean model for home corners and one mean model for away corners. This gives raw point forecasts $\hat{\mu}_{home}$ and $\hat{\mu}_{away}$.
2. Apply the late-window correction to remove recent level bias in the raw mean forecasts.
3. Apply the market-teacher adjustment. This is trained on partition A and applied out of sample on partition B, so the mean head does not directly read the same market it will later be evaluated against in Q2.
4. Fit one variance model for the home side and one for the away side. These use predicted mean, market certainty, and rolling volatility to estimate $\hat{\sigma}^2_{home}$ and $\hat{\sigma}^2_{away}$.
5. Build residual quantiles $q10, q50, q90$ from historically similar matches. Those quantiles give Q2 a lower anchor, a median anchor, and an upper anchor.

At that point the match-level point forecast is already done:

$$\text{Q1 point forecast} = (\hat{\mu}_{home}, \hat{\mu}_{away}).$$

The variance layer does not change those means. It wraps uncertainty around them.

So the final exported Q1 prediction is not just:

$$\hat{\mu}_{home},\ \hat{\mu}_{away}.$$

It is:

$$\hat{\mu}_{home},\ \hat{\mu}_{away},\ \hat{\sigma}^2_{home},\ \hat{\sigma}^2_{away},\ \hat{q}_{10},\ \hat{q}_{50},\ \hat{q}_{90}.$$

That matters because Q2 needs a distribution, not just a point estimate. The mean tells Q2 where the centre is. The variance and quantile anchors tell Q2 how wide the distribution should be and how cautious it should be in the tails.

In the current pipeline, Q2 turns those Q1 outputs into market probabilities in the following way:

1. treat $\hat{\mu}_{home}, \hat{\sigma}^2_{home}$ as the first two moments of the home-corner count;
2. treat $\hat{\mu}_{away}, \hat{\sigma}^2_{away}$ as the first two moments of the away-corner count;
3. convert those moments into a negative-binomial parameterisation for the two team-level count distributions;
4. combine the two team distributions into total-corner and corner-difference distributions, with an extra shared-correlation correction estimated on partition A;
5. use the $q10/q50/q90$ anchors as a guardrail when a quoted market line is far outside Q1's own empirical range.

So the split of responsibilities is clean:

- Q1 predicts the centre and spread of each team's corner count;
- Q2 converts that into probabilities for over/under, handicap, and 1X2 betting markets.

This is why mean and variance are enough for the next step. Once Q1 gives a centre and a spread for the home side, and a centre and a spread for the away side, Q2 can build approximate count distributions for both teams. From those distributions it can compute the probability of total corners going over a line, the probability of home corners beating away corners, and the probability of home, draw, or away in the corner 1X2 market.

Question 2. Using your model from Q1, (a) calculate EV as the expected value of a bet, and (b) how many selections are you betting? Why?
Answer.

I first turn the Q1 outputs into market probabilities.

For the home and away teams, Q1 gives:

$$\hat{\mu}_{home},\ \hat{\sigma}^2_{home},\ \hat{\mu}_{away},\ \hat{\sigma}^2_{away}.$$

From these, the betting layer forms total-corner and corner-difference moments:

$$\mu_{total} = \hat{\mu}_{home} + \hat{\mu}_{away},$$

$$\sigma^2_{total} = \hat{\sigma}^2_{home} + \hat{\sigma}^2_{away} + 2\rho\sqrt{\hat{\sigma}^2_{home}\hat{\sigma}^2_{away}},$$

$$\mu_{diff} = \hat{\mu}_{home} - \hat{\mu}_{away},$$

$$\sigma^2_{diff} = \hat{\sigma}^2_{home} + \hat{\sigma}^2_{away} - 2\rho\sqrt{\hat{\sigma}^2_{home}\hat{\sigma}^2_{away}},$$

where ρ is a shared home-away corner correlation estimated on partition A.

These moments are then converted into market probabilities in two different ways:

- for OU, use the total-corner mean and variance to build a negative-binomial total distribution, then compute $P(T > L)$;
- for HC and 1X2, use the corner-difference mean and variance to approximate a distribution for $D = C_{home} - C_{away}$, then compute the probability of home, draw, or away relative to the quoted line.

For a decimal-odds bet with model win probability p and quoted odds o, the unit-stake expected value is:

$$\text{EV} = p \cdot o - 1.$$

That is the quantity used for filtering.

The raw model probabilities are then calibrated on partition A with per-side isotonic calibration, and the most aggressive positive-EV tails are shrunk back toward no-vig market probabilities when the model looks too confident. The final quantity used for selection is `ev_tail`, not the uncalibrated EV.

Under the current default settings, the selection rule is:

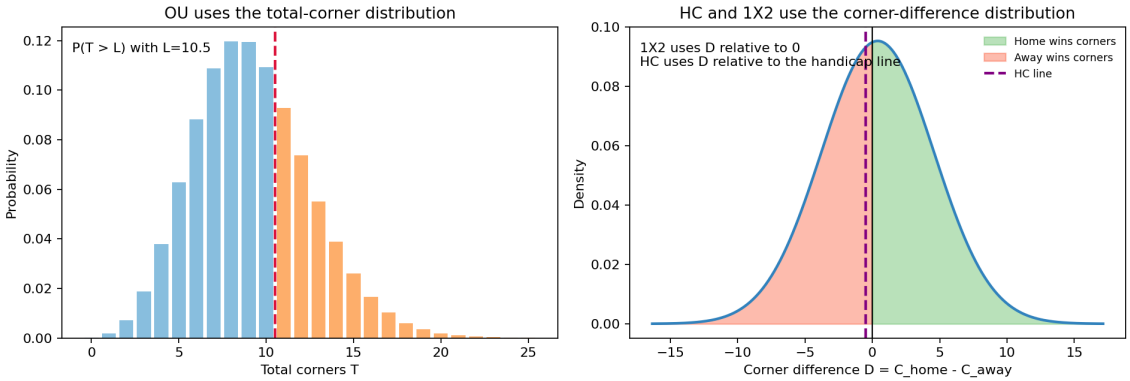

Figure 1: How the model turns Q1 moments into market probabilities: OU uses the total-corner distribution, while HC and 1X2 use the corner-difference distribution.

- bet 1X2 only if EV > 0.03;
- bet HC only if EV > 0.023;
- bet OU only if EV > 0.03.

These thresholds were not chosen arbitrarily. After calibration and tail shrink, 1X2 remained the weakest and noisiest market, so it needed the strictest filter. In the realised partition-B sweep, keeping HC and OU fixed at 0.023 and 0.03, moving the 1X2 threshold from 0.00 to 0.03 improved unit-stake RoI from 18.6% to 35.2%, and at 0.035 the 1X2 book was filtered out completely, leaving only HC and OU. HC was much less sensitive in the 0.02 to 0.03 range, so 0.023 was kept as a middle ground. OU improved steadily from 0.02 to 0.03, so 0.03 was retained as the cleaner setting.

On clean partition B, this produces:

- 866 selections in total;
- 261 from 1X2;
- 456 from HC;
- 149 from OU.

I am not betting every positive-EV line. I am only betting selections that clear the market-specific thresholds after calibration and tail shrink. The reason is practical: the smallest positive-EV bets are exactly where the model tends to be most optimistic, so it is better to give up some volume and keep the average edge cleaner.

Question 3. "Define Stake as the amount that you bet, calculate PnL as the Profit and Loss and RoI as PnL/Stake for each selection. (i) Calculate the overall PnL and RoI. (ii) Plot the cumulative PnL and EV as a function of time in days. (iii) Plot the 2-week rolling PnL and EV average."

Answer. For stake sizing, I use Kelly rather than flat staking. The reason is that Kelly links bet size to estimated edge and current bankroll. More importantly, it comes from maximising expected log wealth rather than raw expected cash profit. That matters because log wealth strongly penalises paths that drive the bankroll close to zero.

For one bet with win probability p, decimal odds o, and stake fraction f, the one-step expected log return is

$$g(f) = p \log\left(1 + f(o-1)\right) + (1-p)\log(1-f).$$

The first-order condition gives the full Kelly fraction

$$f^* = \frac{po - 1}{o - 1}.$$

So Kelly is the fraction implied by maximising expected log growth. Compared with a plain EV rule, the log objective is much harsher on strategies that risk wiping out capital, because $\log(B) \to -\infty$ as bankroll $B \to 0$.

I test three stake policies:

- 100% Kelly: $f = f^*$;
- 50% Kelly: $f = 0.5f^*$;
- 25% Kelly: $f = 0.25f^*$.

The bankroll rules in this section are fixed as:

- initial bankroll = 100 USD;
- fee rate = 3%;
- minimum fee = 0.1 USD per bet;
- maximum single-bet stake = 5000 USD;
- bankroll cannot go below zero;
- once bankroll hits zero, betting stops.

So for one candidate bet at time t, the executable stake is

$$s_t = \min(B_t \cdot f,\ 5000),$$

where B_t is the current bankroll. The fee is

$$\text{fee}_t = \max(0.03 s_t,\ 0.1).$$

If several bets share the same kickoff time, the code rescales them so that the total cash outlay

$$\sum_t (s_t + \text{fee}_t)$$

does not exceed the bankroll available before that kickoff block.

For one realised bet with decimal odds o_t, stake s_t, fee fee_t, and outcome $w_t \in \{0, 1\}$,

$$\text{PnL}_t = \begin{cases} s_t(o_t - 1) - \text{fee}_t, & \text{if } w_t = 1, \\ -s_t - \text{fee}_t, & \text{if } w_t = 0. \end{cases}$$

Overall PnL and turnover RoI are:

$$\text{Total PnL} = \sum_t \text{PnL}_t, \qquad \text{RoI} = \frac{\sum_t \text{PnL}_t}{\sum_t s_t}.$$

Under these exact bankroll constraints on partition B, the results are:

Strategy	Total PnL	Ending Bankroll	Turnover RoI	Max Drawdown	Sharpe-style Proxy
100% Kelly	+1,299,402.01	1,299,502.01	34.39%	−88.35%	5.20
50% Kelly	+1,249,139.57	1,249,239.57	36.32%	−44.17%	5.32
25% Kelly	+1,073,074.66	1,073,174.66	37.77%	−22.09%	7.82

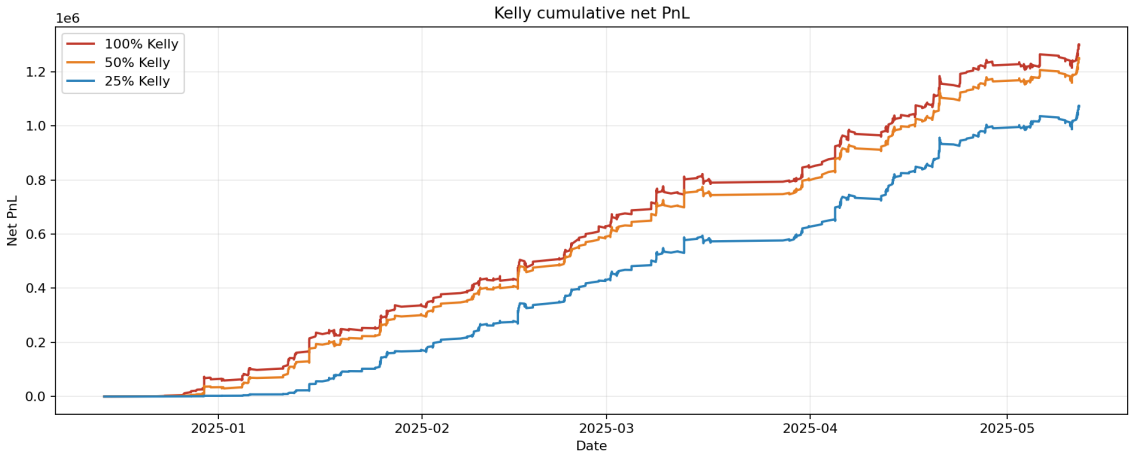

Figure 2: Cumulative net PnL for 100%, 50%, and 25% Kelly under the bankroll, fee, and stake-cap constraints.

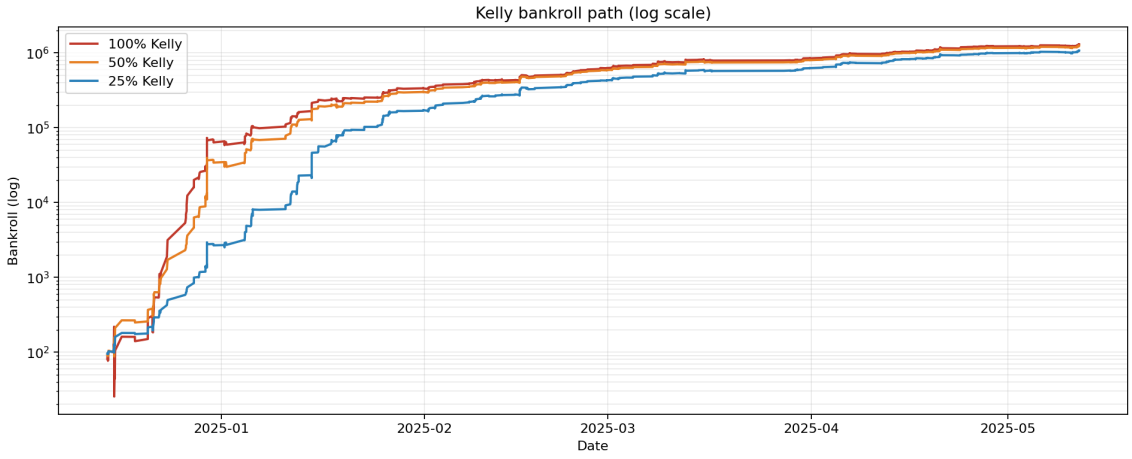

Figure 3: Kelly bankroll paths on a log scale. This view is more readable because the three strategies end at very different bankroll levels.

None of the three Kelly paths went bust in this realised sample, but full Kelly has extremely severe drawdowns. The most balanced line here is 25% Kelly: lower terminal PnL than full Kelly, but the highest turnover RoI, the mildest drawdown, and the best simple Sharpe-style proxy based on daily bankroll returns. That is why I would prefer 25% Kelly in practice. The point is not to maximise the paper terminal value at all costs. The point is to maximise risk-adjusted return.

Question 4. "Do you think that your assumed EV is an accurate representation of your real edge? (Hint: Run a Monte Carlo simulation)"
Answer. No. The assumed EV is a model output, not a ground truth edge. Kelly changes the stake size, but it does not repair probability miscalibration by itself. If the probability model is shifted, overconfident, or unstable, the real edge can be much smaller than the paper EV, and it can even turn negative.

To test this, I run a Monte Carlo simulation on the selected bets. For each selected bet i, I treat the calibrated tail probability p_i as the Bernoulli win probability, simulate the bet outcome many times, and sum the simulated PnL across the whole portfolio. Repeating that gives a distribution for total PnL under the model's own assumptions.

If the model's EV were perfectly calibrated, the realised PnL would sit in a fairly ordinary part of that simulated distribution. If the realised PnL kept landing near the extreme lower tail, that would mean the model is overstating its edge.

The probability model behind the Kelly sizing was checked earlier with Monte Carlo on the selected partition-B bets:

- actual total PnL = +305.09;
- Monte Carlo median PnL = +249.61;
- Monte Carlo 5th to 95th percentile range = [+181.73, +319.25];
- actual percentile = 90th.

So the realised outcome is better than the model median in this sample. That is encouraging, but it still does not prove that the EV is accurate. The Monte Carlo check is only an internal consistency test. It asks: if the model probabilities were correct, would the realised PnL look plausible? It does not establish that the probabilities are stable out of sample.

To make the stress-test idea more concrete, I also plot one pessimistic scenario. In that scenario, I cut the model edge by shrinking each selected probability 20% toward the no-vig market probability:

$$p_{\text{stress}} = p_{\text{market}} + 0.8\,(p_{\text{tail}} - p_{\text{market}})\,.$$

Under that stressed probability set, the Monte Carlo median falls to +199.32, which sits around the 11th percentile of the original model distribution. This is a useful visual reminder that the current edge can look much less comfortable once the distribution is perturbed even mildly.

The more serious failure mode is distribution shift. If the home and away means drift, if the variance layer becomes too narrow, if the tails are mis-specified, or if the bookmaker market adapts, then the realised win rate can fall below the model probability. In that case Kelly can lose money fast because the stake is a direct function of estimated edge.

That is why stress testing matters. Reasonable stress tests here are:

- shrink model probabilities toward market-implied probabilities;
- push the mean predictions away from their fitted values;
- widen or narrow the variance inputs;
- raise fees or lower the maximum bet size;
- remove the highest-EV tail and see how much of the PnL survives.

If the strategy only works under the exact fitted distribution, then the edge is too fragile.

There is also a market-microstructure risk that the backtest does not solve: if a bookmaker detects an advantage player, it can lower the bet cap, delay acceptance, or restrict the account entirely. In that case the theoretical Kelly fraction is no longer deployable, so realised growth can be much lower than the paper path even if the raw EV estimate is correct.

This is also why I prefer showing 25% and 50% Kelly alongside full Kelly rather than trusting full Kelly blindly. If the EV estimate is even slightly too high, full Kelly overreacts immediately. Fractional Kelly gives more room for model error.

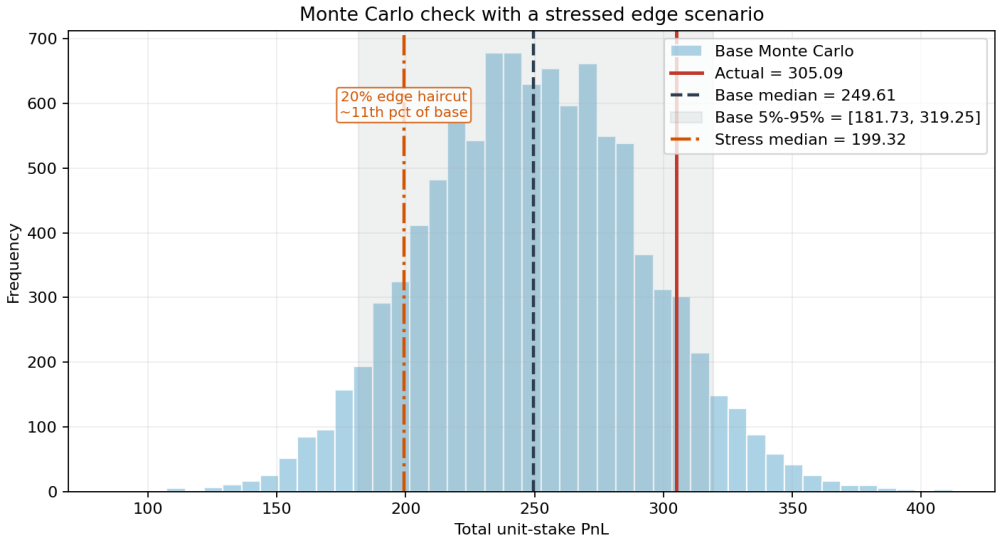

Figure 4: Monte Carlo distribution of unit-stake total PnL for the selected partition-B bets, with one stressed scenario overlaid. The stressed line uses a 20% shrink toward market-implied probabilities to show how a mild edge haircut moves the portfolio into the lower part of the original model distribution.